

Documento de Diseño Arquitectónico

NexaSafe — Componentes, flujo de datos y modelo de datos. Describe la arquitectura de software, sus decisiones y el porqué detrás de cada una.

22 sep 2026

FECHA DE ENTREGA (V1.0)

3 oct 2026

REVISIÓN V1.1

Expo + React Native
MÓVILSupabase
BACKEND

1 Introducción y objetivos de diseño

Describe la arquitectura de software de NexaSafe: sus componentes, el flujo de datos entre ellos, el modelo de datos que sostiene el dominio y las decisiones de diseño que explican por qué el sistema está construido como está.

01 · PRIORIDAD MÁXIMA

Seguridad y privacidad por defecto

El sistema trata datos sensibles de menores; cada decisión de arquitectura parte de minimizar exposición.

02

Disponibilidad en red pobre

El trayecto real ocurre en la calle, no en un entorno controlado.

03

Velocidad de entrega

Equipo de 2–3 personas, cero infraestructura propia que mantener.

04

Auditabilidad

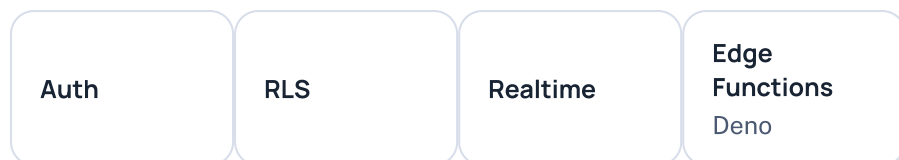
Todo incidente debe poder reconstruirse después de ocurrido.

2 Vista de componentes

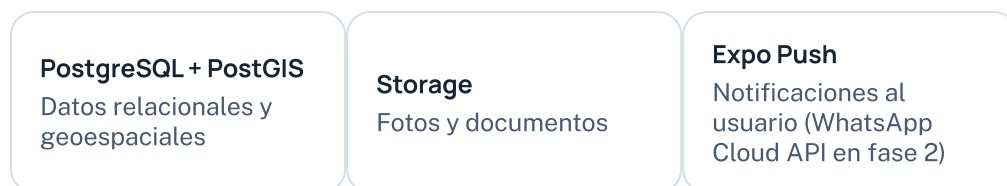
App móvil

Expo / React Native / TypeScript
— Protegido · Guardián · Puesto de control

↓ HTTPS + Realtime (WSS)



Supabase — audit log append-only · políticas por fila y por rol



La app móvil es un único binario con tres experiencias condicionadas por el rol autenticado (protegido, guardián o puesto de control), no tres aplicaciones separadas. Supabase concentra autenticación, autorización (RLS), tiempo real y lógica de servidor (Edge Functions) sin que el equipo administre infraestructura propia. PostgreSQL con PostGIS resuelve las consultas geoespaciales del corredor seguro y las geocercas; Storage custodia fotos y documentos; Expo Push es el canal de notificación de la fase 1, y WhatsApp Cloud API se suma como canal secundario en la fase 2.

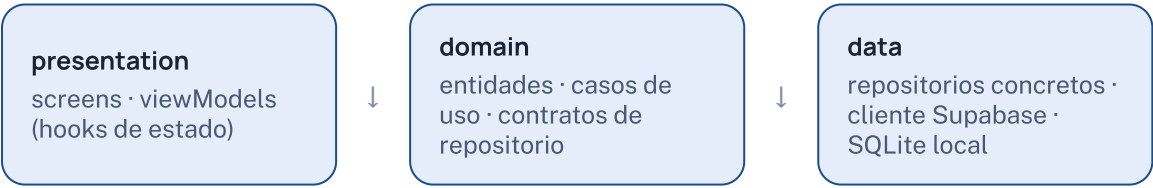
3 Stack tecnológico

CAPA	TECNOLOGÍA	JUSTIFICACIÓN
Móvil	Expo SDK + React Native + TypeScript	Requisito de la asignatura; expo-location, expo-taskmanager, expo-secure-store cubren ubicación en segundo plano y almacenamiento cifrado.
Arquitectura móvil	Feature-based: src/features/<f>/{data, domain, presentation} + src/core	Continuidad con la estructura ya definida en el proyecto.
Backend	Supabase — PostgreSQL gestionado + Auth + Realtime + Edge Functions (Deno/TypeScript)	Elimina el montaje, despliegue y mantenimiento de infraestructura propia; autenticación, autorización por RLS y tiempo real vienen resueltos.
Base de datos	PostgreSQL 17 con extensión PostGIS (versión con la que Supabase crea los proyectos)	Consultas geoespaciales para corredor seguro y geocercas.
Tiempo real	Supabase Realtime (WebSocket sobre Postgres)	Difusión de posición a N guardianes sin polling ni servidor propio.

CAPA	TECNOLOGÍA	JUSTIFICACIÓN
Persistencia local	expo-secure-store + caché local de la ruta (fase 1) · SQLite / expo-sqlite (fase 2)	expo-secure-store custodia la sesión (tokens). El PIN nunca se guarda en el dispositivo. La ruta guardada se conserva en caché local (E3-01); como revela la rutina del menor, el mecanismo y su cifrado se definen en el refinement del Sprint 2. SQLite sostiene la cola offline de E10 en la fase 2.
Gráficos	react-native-gifted-charts	Gráfico interactivo de alertas por día y por tipo de cierre en la vista institucional (E8-01). Recharts y D3 no corren en React Native.
Notificaciones	Expo Push Notifications (fase 1) + WhatsApp Cloud API de Meta (fase 2)	Push como canal principal; WhatsApp a contactos de seguridad y puesto de control vía plantilla aprobada.
Infraestructura	Supabase CLI (migraciones versionadas) · GitHub Actions · EAS Build	Reproducibile y suficiente para el alcance académico, sin contenedores propios.

4 Arquitectura móvil

Organización por características (*feature-based*) con separación en tres capas dentro de cada feature, inspirada en Clean Architecture / MVVM. Ya está en uso en el prototipo actual del repositorio (feature auth) y se extiende al resto de features del dominio.



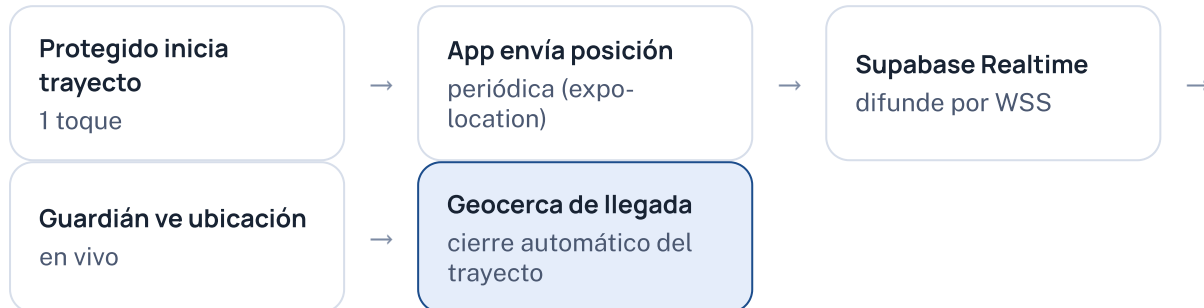
src/core — api · storage · permissions · theme · utils — compartido entre todas las features

CAPA	RESPONSABILIDAD
domain	Entidades del negocio y contratos (interfaces de repositorio); no conoce Supabase ni React Native.
data	Acceso a datos: repositorios concretos que implementan los contratos de domain contra Supabase o SQLite local.
presentation	Pantallas y view-models; consume casos de uso de domain, nunca accede a data directamente.
core	Infraestructura transversal: cliente API, almacenamiento seguro, permisos del dispositivo, tema visual y utilidades.

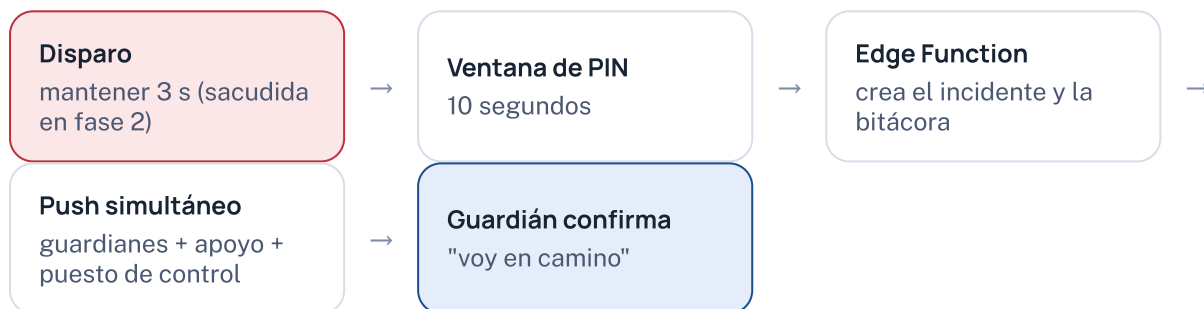
5 Flujo de datos

Tres flujos de datos críticos: trayecto acompañado, alerta de pánico y operación sin conexión. Corresponden a los casos de uso CU-02, CU-04/CU-05 y a la épica E10 (fase 2).

5.1 Trayecto acompañado con seguimiento en vivo

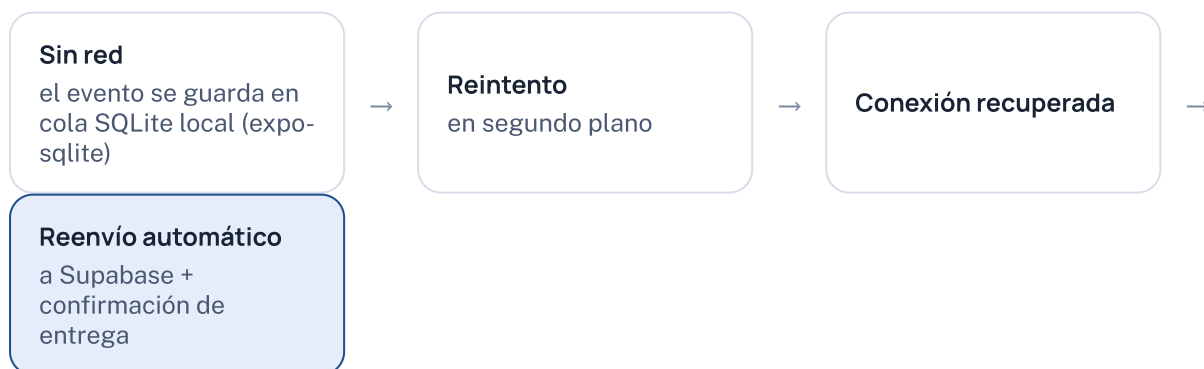


5.2 Alerta de pánico



↳ PIN correcto en la ventana de 10 s (verificado por la Edge Function `auth-pin`) → la alerta se cancela; nadie es notificado.
PIN incorrecto o temporizador vencido → la alerta se envía. Sin conexión el PIN no se puede verificar y la alerta no se cancela.

5.3 Operación degradada sin conexión Fase 2 · E10



6 Modelo de datos

Modelo entidad-relación simplificado del dominio. No incluye tablas puramente técnicas; se centra en las entidades de negocio necesarias para sustentar los requisitos funcionales.



6.1 Diccionario de datos resumido

ENTIDAD	NOTAS DE DISEÑO
Roles	El rol del usuario (protegido, guardian, apoyo, institucion) se guarda en <code>app_metadata.rol</code> de Supabase Auth, que solo el servidor puede escribir; las políticas RLS lo leen del JWT.
guardianes	<code>id</code> es el mismo identificador de <code>auth.users</code> . Las credenciales las gestiona Supabase Auth: no existe <code>password_hash</code> propio.
<code>personal_institucion</code>	Cuentas del puesto de control; <code>id</code> es el de <code>auth.users</code> . Un colegio puede tener varias; <code>activo = false</code> retira el acceso sin borrar el historial. RLS: solo el personal activo de un colegio ve y valida los protegidos de ese colegio.
<code>protegidos.pin_hash</code>	Hash Argon2id del PIN. Solo lo lee y verifica la Edge Function <code>auth-pin</code> (rate limiting), que entrega la sesión del protegido; el mecanismo exacto de sesión se detalla en el refinement de E1-04.
<code>protegidos.estado</code>	Máquina de estados: PENDIENTE_VALIDACION → ACTIVO → INACTIVO. Solo el personal activo del colegio puede mover a ACTIVO (RF-03).
consentimientos	Append-only (RF-38): registra versión de la política, actor y <code>aceptado_en</code> con timestamp de servidor. Una revocación se registra como fila nueva; ninguna fila se actualiza ni se borra.
<code>trayectos.estado</code>	Máquina de estados: EN_CURSO → CERRADO en la fase 1. DESVIADO, ALERTA (por desvío) y VENCIDO quedan definidos para la fase 2 (E5).
<code>alertas.tipo</code>	Enumera el origen: MANUAL_BOTON (fase 1), MANUAL_SACUDIDA y AUTOMATICA_DESVIO (fase 2).
<code>alertas.estado</code>	Enumera el desenlace: ENVIADA, CANCELADA, ATENDIDA, CERRADA.
<code>eventos_incidente</code>	Tabla append-only; ninguna fila se actualiza ni se borra (RF-32). Sustenta la bitácora auditable.
<code>audit_log</code>	Cross-cutting: registra todo acceso de lectura o escritura a datos de un protegido, con actor, acción y timestamp de servidor.
Retención	Trayectos con más de 90 días se purgan automáticamente (RF-39, fase 2); <code>eventos_incidente</code> se conserva 1 año.

7 Seguridad de la arquitectura

CONTROL	IMPLEMENTACIÓN
Cifrado en tránsito	TLS 1.3 obligatorio + certificate pinning en el cliente móvil.
Cifrado en reposo	Cifrado a nivel de columna para documento, foto y ubicación histórica.
Autenticación	JWT de vida corta (15 min) + refresh token rotativo revocable.

CONTROL	IMPLEMENTACIÓN
Autorización	RBAC: protegido, guardian, apoyo, institucion — rol en app_metadata (solo escribible por el servidor), verificado por RLS y en cada Edge Function.
Protección del PIN	Hash Argon2id en protegidos.pin_hash, nunca en texto plano ni en el dispositivo; verificado solo por la Edge Function auth-pin con rate limiting. En el dispositivo, expo-secure-store custodia únicamente la sesión.
Anti-abuso	Rate limiting por IP y por usuario en endpoints de alerta y autenticación.
Anti-tampering	Detección de root/emulador en modo release.
Minimización	Solo se persiste ubicación con trayecto o alerta activos; fuera de eso no se recolecta.
Trazabilidad	Todo acceso a datos de un menor queda en audit_log con actor, acción y timestamp.

8 Decisiones arquitectónicas clave

DECISIÓN	RAZONAMIENTO
Backend en Supabase en lugar de propio	Un equipo de 2–3 personas no puede sostener un backend a medida (FastAPI + Redis + Docker) con la misma calidad de seguridad que una plataforma gestionada con Auth, RLS y Realtime ya endurecidos.
Vista institucional en la misma app, sin dashboard web	Reduce la superficie de código a mantener y de ataque a asegurar; el puesto de control necesita mapa y lista de alertas, no un panel administrativo completo.
WhatsApp Cloud API como canal secundario	El número de prueba solo soporta 5 destinatarios verificados (riesgo R5); el push permanece como canal principal sin ese límite.
Disparo de pánico por sacudida, no botón físico	Evita depender de hardware adicional que el usuario deba comprar y llevar; se limita a modo trayecto con la app en primer plano para reducir falsos positivos.
Escalamiento a la autoridad simulado	La integración real con la línea 123 requiere convenio interinstitucional fuera del alcance académico; el sistema genera y registra el reporte sin transmitirlo a un organismo real (E8-02, fase 2).
Credenciales solo en Supabase Auth	Duplicar contraseñas en una tabla propia (password_hash) crearía una segunda superficie de ataque; guardianes y personal_institucion referencian auth.users.
PIN verificado en el servidor	Supabase Auth no ofrece login por PIN. Verificarlo en una Edge Function permite rate limiting y evita que el hash viaje o se guarde en el teléfono.
Tabla personal_institucion	Permite varios responsables por colegio y desactivarlos sin borrar historial, lo que mitiga R3 (colegio sin responsable).
GitHub Flow (solo main)	Solo existen los ambientes dev y staging; main despliega a staging y las versiones se publican por tag (vX.Y.Z) con EAS Build. Una rama develop no tendría ambiente propio y duplicaría PR.

9 Estructura del repositorio

```
nexasafeapp/
├── .github/
│   ├── workflows/          # ci.yml, nightly-security.yml, release.yml
│   ├── ISSUE_TEMPLATE/     # user-story.md, bug.md, security-finding.md
│   └── pull_request_template.md
├── apps/
│   └── mobile/              # Expo + React Native + TypeScript
│       ├── src/
│       │   ├── core/        # api, storage, permissions, theme, utils
│       │   └── features/
│       │       ├── auth/     {data, domain, presentation}
│       │       ├── profile/
│       │       ├── routes/
│       │       ├── tracking/
│       │       ├── alerts/
│       │       ├── network/
│       │       ├── institution/ # vista del puesto de control
│       │       └── incidents/
│       └── supabase/
│           ├── migrations/   # esquema versionado
│           ├── functions/    # Edge Functions (Deno): auth-pin, notify-push · fase 2: notify-wha
│           └── policies/     # políticas RLS documentadas y probadas
├── docs/
│   ├── entregables/         # ing-software-1/: los 5 documentos oficiales (PDF + versión inter
│   ├── architecture/
│   ├── security/            # threat-model.md, risk-register.md, exceptions.md
│   ├── privacy/             # pia.md, politica-tratamiento.md
│   └── scrum/                # sprint-XX/{planning, review, retro, resumen}.md
├── .pre-commit-config.yaml
├── .gitleaks.toml
├── semgrep.yml
├── CHANGELOG.md
└── README.md
```